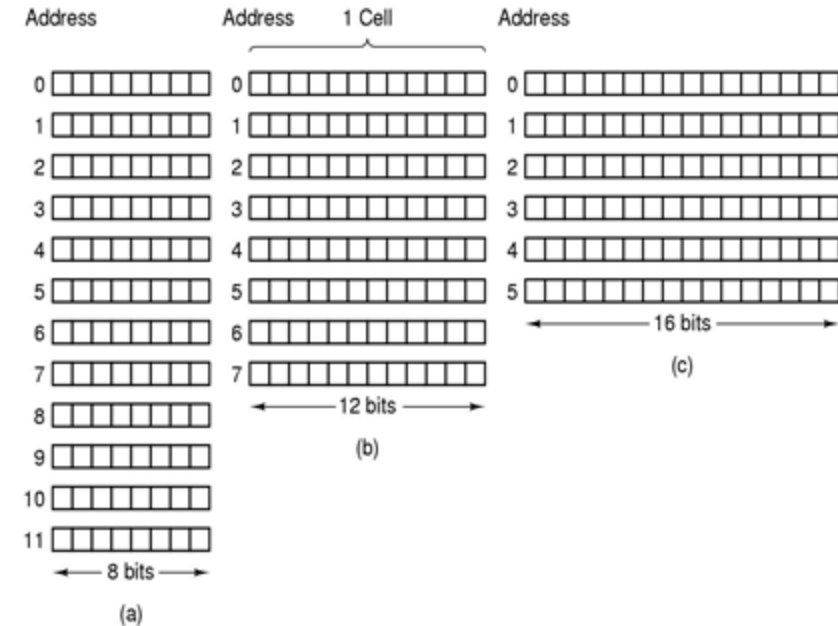


Memoria: Organizzazione e gerarchia

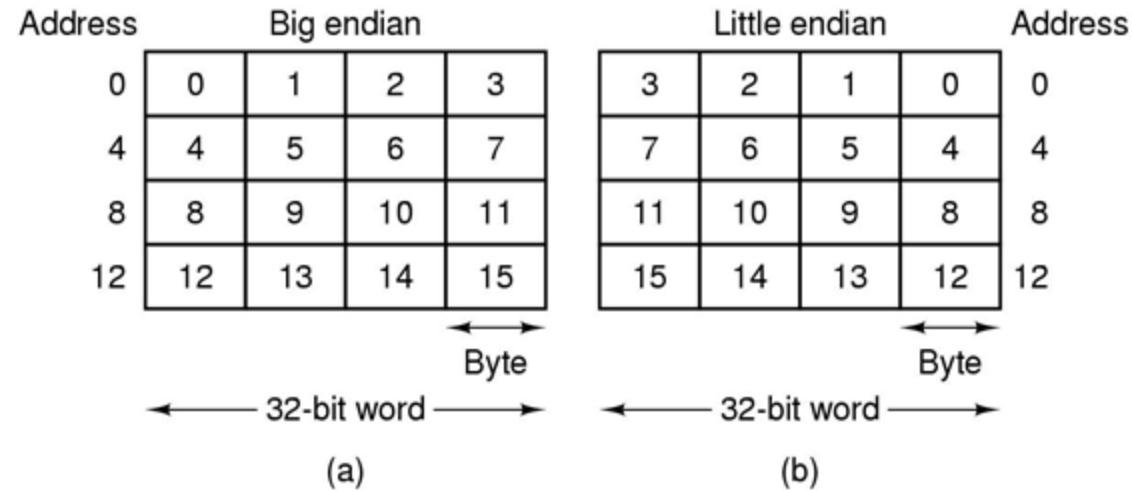
La memoria principale

- Store o storage
- Memoria come insieme di celle (o locazioni)
- Tutte le celle hanno stessa dimensione
- Ogni cella è identificata da un indirizzo
- Se un calcolatore ha n celle allora gli indirizzi vanno da **0** a **$n-1$ indipendentemente** dalla dimensione della cella
- Il numero di bit nell'indirizzo determina il numero massimo di celle indirizzabili



- La cella è la minima unità indirizzabile
- Recentemente dimensione standard: 8 bit (byte), 4 o 8 byte definiscono una **parola** (unità di manipolazione per le istruzioni)

Ordinamento dei byte



(a) **big endian**: la numerazione va da sinistra verso destra:
big end first (SPARC, mainframe IBM)

(b) **little endian**: la numerazione va da destra verso sinistra:
little end first (Intel, RISC-V)

CPU e memoria

CPU più veloci delle memorie: quando la CPU effettua una richiesta alla memoria, essa non ottiene la parola desiderata se non dopo molti cicli di CPU.

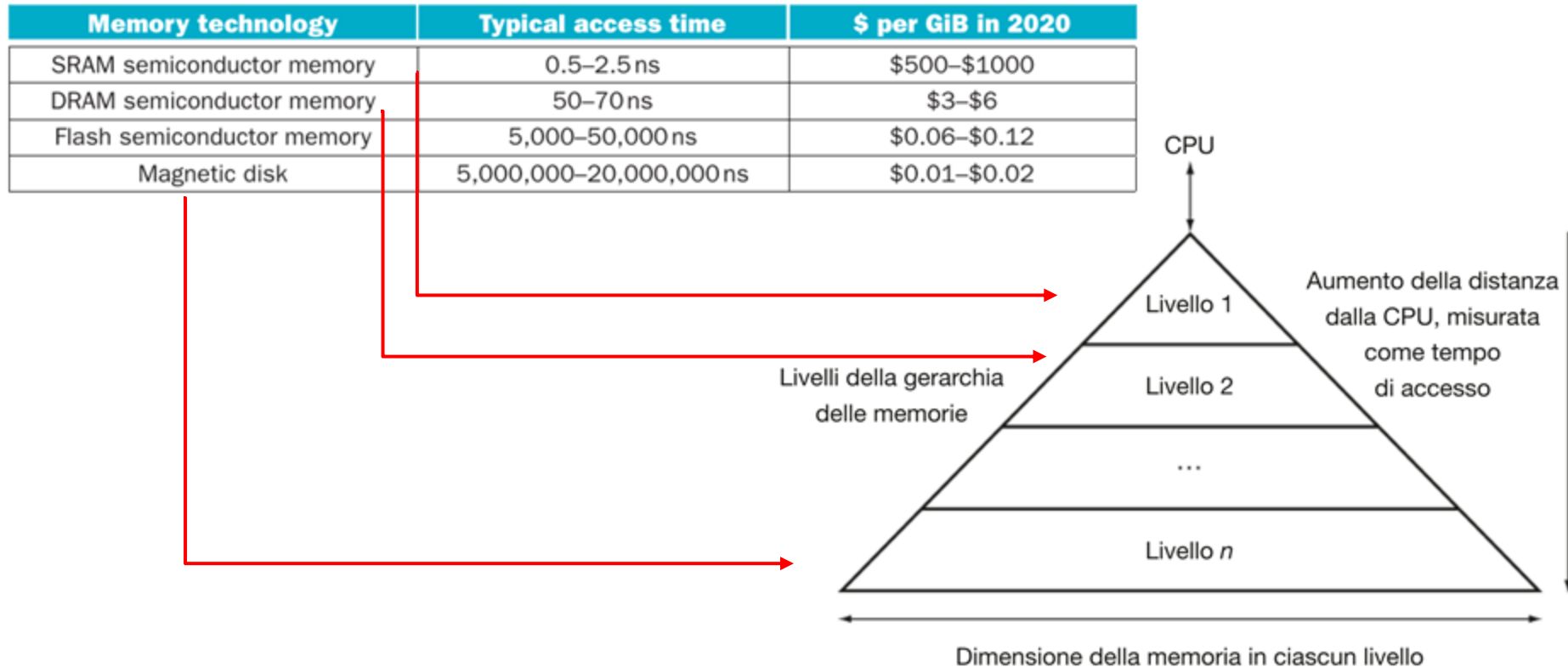
Piccola quantità di memoria veloce (costosa) o grande quantità di memoria più lenta?



Tecniche che permettono di combinare le due soluzioni per ottenere a un prezzo ragionevole sia la velocità sia la notevole capacità

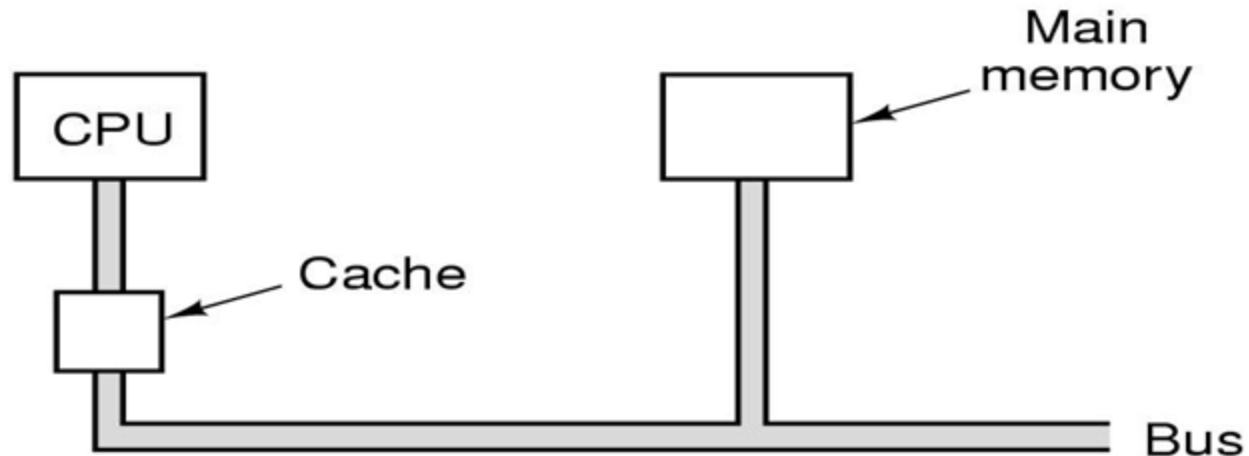
Memoria piccola e veloce: **cache**

Gerarchia della memoria



- Gerarchia della memoria secondo la capacità di memorizzazione
- Questa gerarchia rispetta i costi di memorizzazione (più in basso i più economici, più in alto i più costosi)
- Rispetta anche i tempi di accesso all'informazione

Memoria cache



- L'uso della memoria di cache è basata sul ***principio di località*** del codice: i programmi non accedono alla memoria a caso
- Le parole di memoria più usate vengono tenute in una cache

Località Temporale

Principio secondo cui se si accede a una determinata locazione di memoria, è molto probabile che vi si acceda di nuovo dopo poco tempo.

fib:

```
beq  a0, zero, fib_end
addi t0, zero, 1
beq  a0, t0, fib_end
```

```
# general case n > 1
```

```
addi sp, sp, -8
sw   ra, 0(sp)
sw   a0, 4(sp)
```

```
addi a0, a0, -1
jal  fib
lw   t0, 4(sp)
sw   a0, 4(sp)
addi a0, t0, -2
jal  fib
```

```
lw   t0, 4(sp)
add  a0, a0, t0
lw   ra, 0(sp)
addi sp, sp, 8
```

fib_end:

```
ret
```

Sia i dati che le istruzioni possono essere presenti nella cache !

Località Spaziale

principio secondo cui se si accede a una determinate locazione di memoria, è molto probabile che si acceda alle locazioni vicine ad essa dopo poco tempo.

```
loop:
  beq t1, a1, endloop      # while(i < size)
  lw t2, 0(a0)             # array[i]

  bge t2, a2, carryon      # if(array[i] < min_val)
  mv t0, t1                # min_idx = i;
  mv a2, t2                # min_val = array[i];

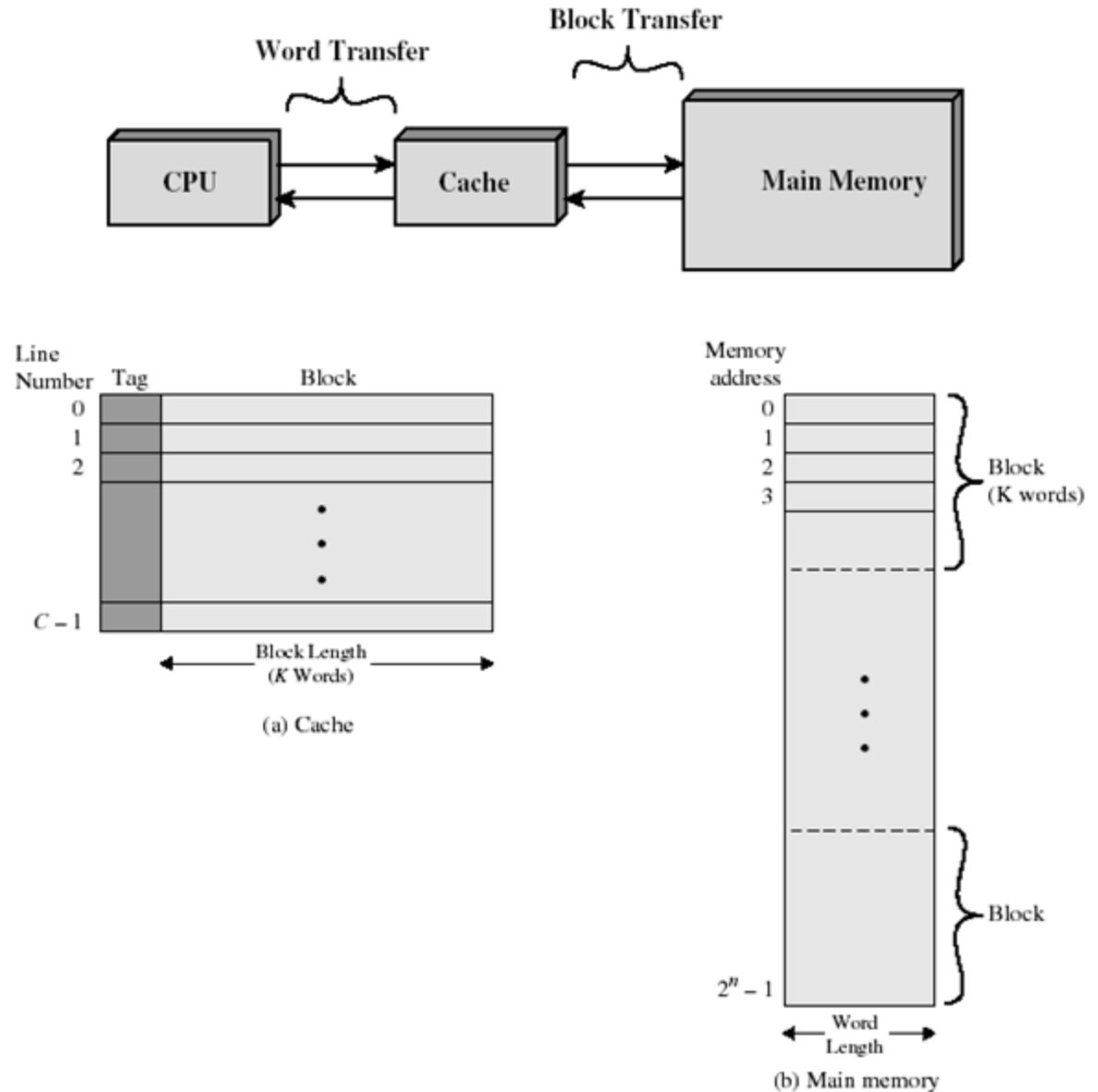
  carryon:
    addi a0, a0, 4          # spostato il puntatore alla prossima word
    addi t1, t1, 1          # ++i;
    j loop

endloop:
  mv a0, t0
  jr ra
```

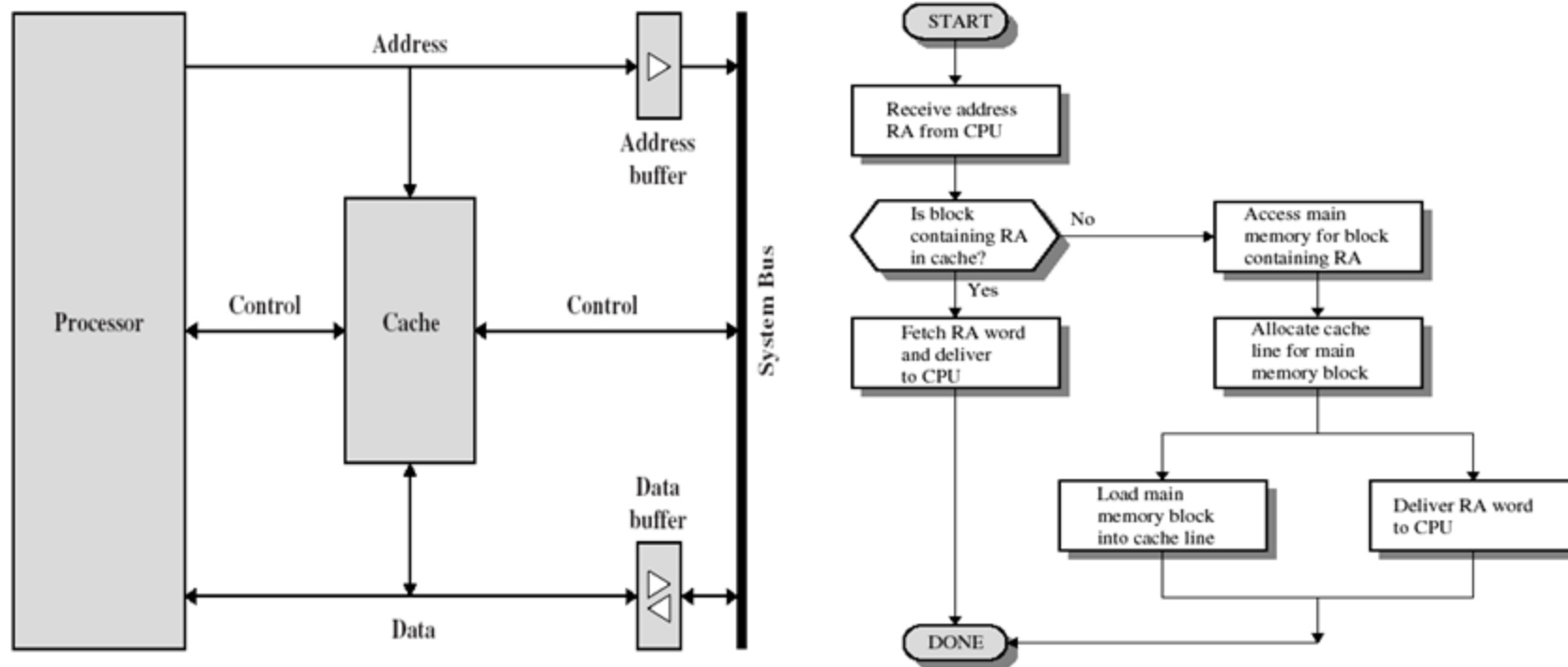
Sia i dati che le istruzioni possono essere presenti nella cache !

Memoria cache

- La memoria principale consiste di 2^n byte indirizzabili
- La memoria può essere considerata anche organizzata in **blocchi di k byte** ($M = 2^n/k$ blocchi)
- La cache consiste di C linee di k byte (blocchi)
- $C \ll M$
- Si usano blocchi per il principio di località



Memoria cache



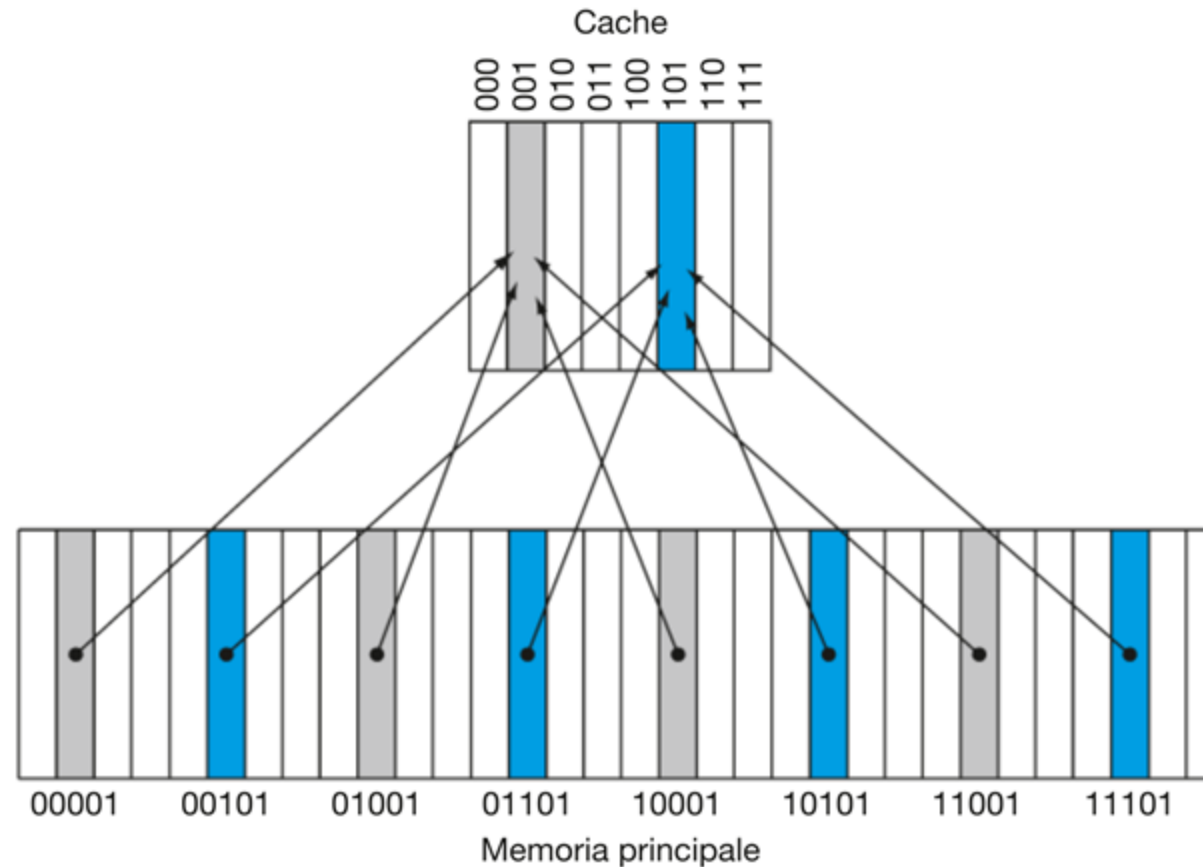
- Se la parola è contenuta nella cache questa è inviata direttamente alla CPU (**hit / hit rate / tempo di hit**)
- Altrimenti il blocco contenente la parola cercata è caricata nella cache e inviata nello stesso tempo alla CPU (**miss della cache / miss rate / penalità di miss**)

Memoria cache

Problemi relativi alla progettazione della memoria cache:

- **Dimensione:** maggiore dimensione implica migliori prestazioni, ma maggiore costo
- **Dimensione della linea di cache**
(in una cache di 14 KB è meglio avere 1024 linee di 16 byte o 2048 di 8 byte?)
- **Come tener traccia** di quali parole di memoria sono memorizzate nella cache in un dato momento?
- Istruzioni e dati in una sola cache “**unificata**” o è preferibile avere cache “**specializzate**” per dati e istruzioni (architettura Harvard), in modo da parallelizzare l’accesso alle istruzioni e ai dati?
- **Numero delle cache:** oggi anche tre livelli di cache

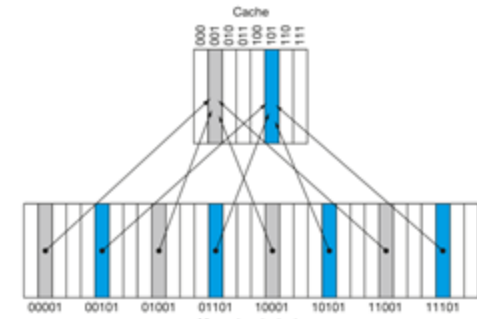
Mappatura diretta



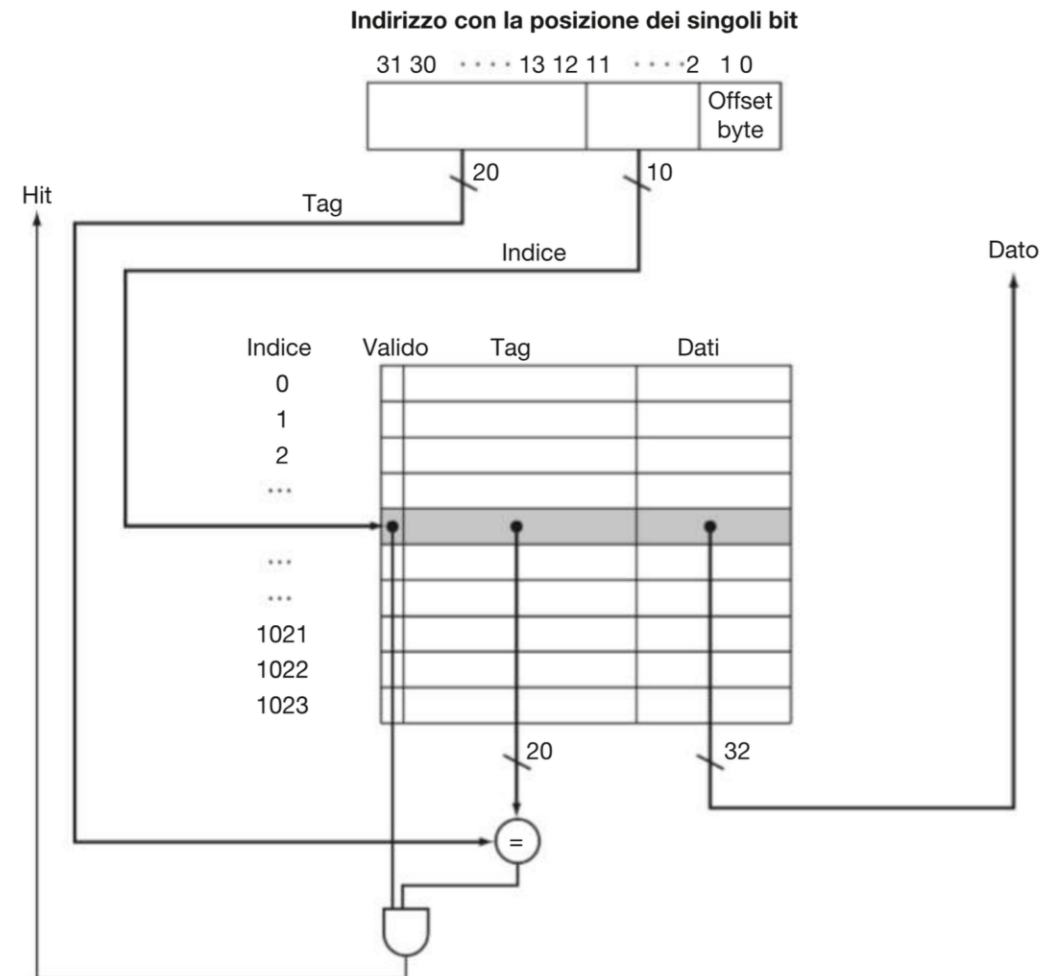
- Una cache in cui ad ogni locazione della memoria principale corrisponde una (e una sola) locazione della cache
Nella figura: 1 blocco = 1 parola, in pratica blocchi da k byte

(Indirizzo del blocco) modulo (numero di **blocchi** nella cache)

Mappatura diretta



- Come si capisce se
 - In una linea è presente un dato?
 Bit di validità
 - Nella cache è presente il dato ricercato?
- tag



(Indirizzo del blocco) modulo (numero di **blocchi** nella cache)

Mappatura diretta

Un esempio

- Cache a mappatura diretta di otto blocchi
- I tre bit meno significativi indicano il numero della linea
- Elenco di nove richieste

Indirizzo decimale del dato nella memoria principale	Indirizzo binario del dato nella memoria principale	Hit o miss nell'accesso alla cache	Blocco della cache corrispondente (dove trovare o scrivere il dato)
22	10110 _{due}	Miss (5.9b)	$(10110_{\text{due}} \bmod 8) = 110_{\text{due}}$
26	11010 _{due}	Miss (5.9c)	$(11010_{\text{due}} \bmod 8) = 010_{\text{due}}$
22	10110 _{due}	Hit	$(10110_{\text{due}} \bmod 8) = 110_{\text{due}}$
26	11010 _{due}	Hit	$(11010_{\text{due}} \bmod 8) = 010_{\text{due}}$
16	10000 _{due}	Miss (5.9d)	$(10000_{\text{due}} \bmod 8) = 000_{\text{due}}$
3	00011 _{due}	Miss (5.9e)	$(00011_{\text{due}} \bmod 8) = 011_{\text{due}}$
16	10000 _{due}	Hit	$(10000_{\text{due}} \bmod 8) = 000_{\text{due}}$
18	10010 _{due}	Miss (5.9f)	$(10010_{\text{due}} \bmod 8) = 010_{\text{due}}$
16	10000 _{due}	Hit	$(10000_{\text{due}} \bmod 8) = 000_{\text{due}}$

Mappatura diretta

Un esempio

Indice	V	Tag	Dati
000	N		
001	N		
010	N		
011	N		
100	N		
101	N		
110	N		
111	N		

a. Lo stato iniziale della cache dopo l'accensione del calcolatore

Indice	V	Tag	Dati
000	N		
001	N		
010	N		
011	N		
100	N		
101	N		
110	S	10 _{due}	Memoria (10110 _{due})
111	N		

b. Dopo avere gestito una miss all'indirizzo (10110_{due})

Indice	V	Tag	Dati
000	N		
001	N		
010	S	11 _{due}	Memoria (11010 _{due})
011	N		
100	N		
101	N		
110	S	10 _{due}	Memoria (10110 _{due})
111	N		

c. Dopo avere gestito una miss all'indirizzo 11010_{due}

Indice	V	Tag	Dati
000	S	10 _{due}	Memoria (10000 _{due})
001	N		
010	S	11 _{due}	Memoria (11010 _{due})
011	N		
100	N		
101	N		
110	S	10 _{due}	Memoria (10110 _{due})
111	N		

d. Dopo avere gestito una miss all'indirizzo 10000_{due}

Mappatura diretta

Un esempio

Indice	V	Tag	Dati
000	S	10 _{due}	Memoria (10000 _{due})
001	N		
010	S	11 _{due}	Memoria (11010 _{due})
011	S	00 _{due}	Memoria (00011 _{due})
100	N		
101	N		
110	S	10 _{due}	Memoria (10110 _{due})
111	N		

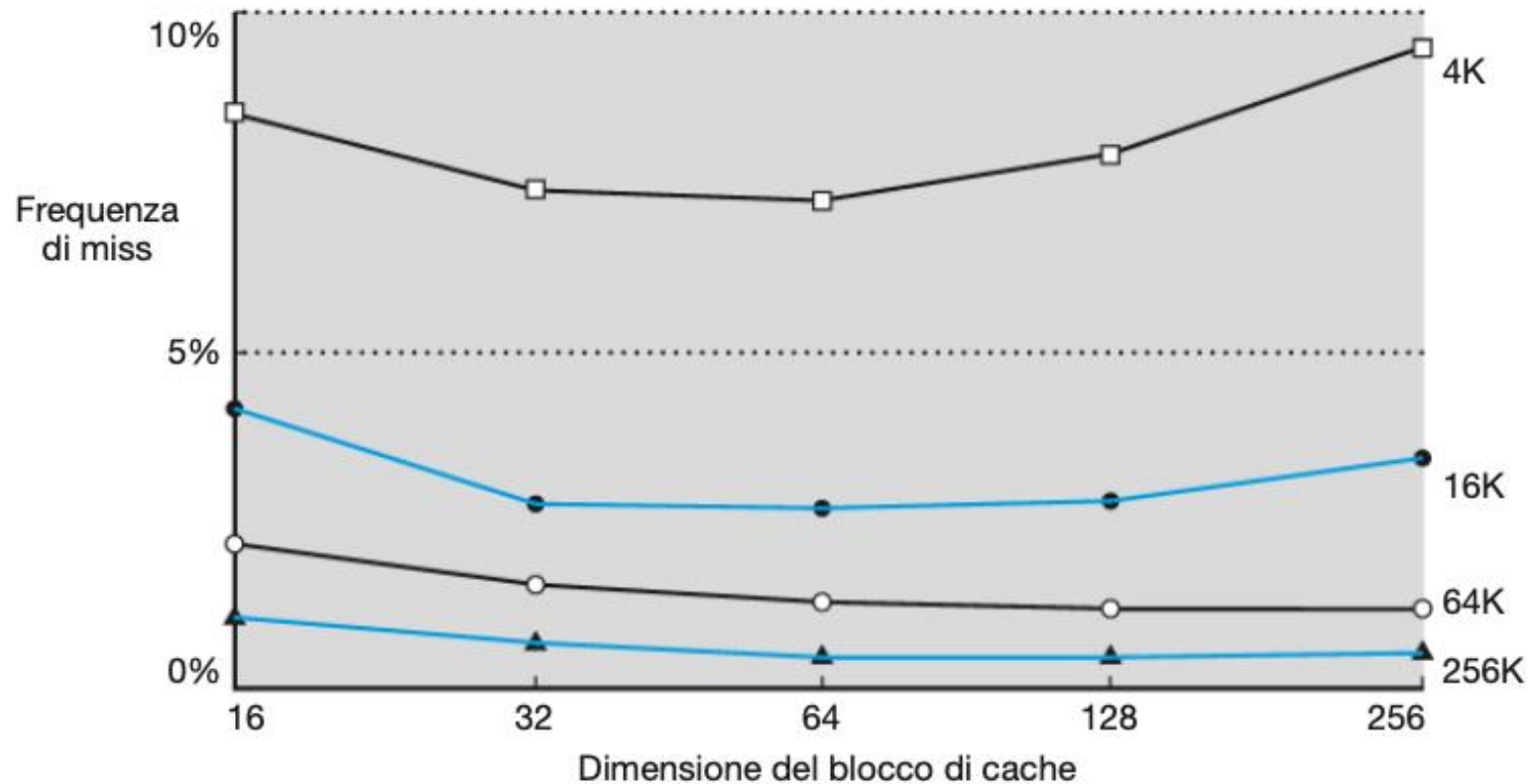
e. Dopo avere gestito una miss all'indirizzo 00011_{due}

Indice	V	Tag	Dati
000	S	10 _{due}	Memoria (10000 _{due})
001	N		
010	S	10 _{due}	Memoria (10010 _{due})
011	S	00 _{due}	Memoria (00011 _{due})
100	N		
101	N		
110	S	10 _{due}	Memoria (10110 _{due})
111	N		

f. Dopo avere gestito una miss all'indirizzo 10010_{due}

Prestazioni di una cache

Aumentare la dimensione del blocco non è sempre una buona idea



Completamente associativa

Un blocco della memoria principale può essere scritto in una qualsiasi locazione della cache

La ricerca va effettuata su tutti gli elementi della cache

- Svolta in parallelo attraverso un comparatore (aumentando i costi)

Adatta solo per un numero ridotto di blocchi



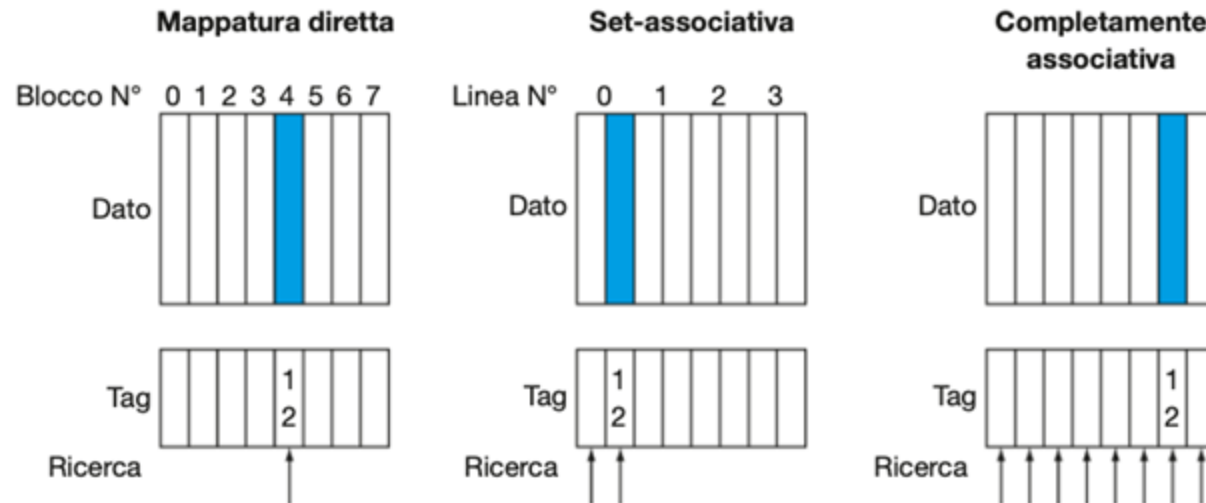
Cache set-associativa

Schema intermedio

Ogni linea può contenere più blocchi (numero di vie)

Individuata la linea, il blocco viene ricercato all'interno degli slot della linea

(Numero del blocco) modulo (Numero delle **linee** della cache)



Cache set-associativa

Cache set-associativa a una via

(a mappatura diretta)

Blocco	Tag	Dato
0		
1		
2		
3		
4		
5		
6		
7		

Cache set-associativa a due vie

Linea	Tag	Dato	Tag	Dato
0				
1				
2				
3				

Cache set-associativa a quattro vie

Linea	Tag	Dato	Tag	Dato	Tag	Dato	Tag	Dato
0								
1								

Tag: l'informazione per capire se il blocco contiene la parola richiesta

Utilizzo meno recente (LRU):
il blocco sostituito è quello che è rimasto inutilizzato per più tempo

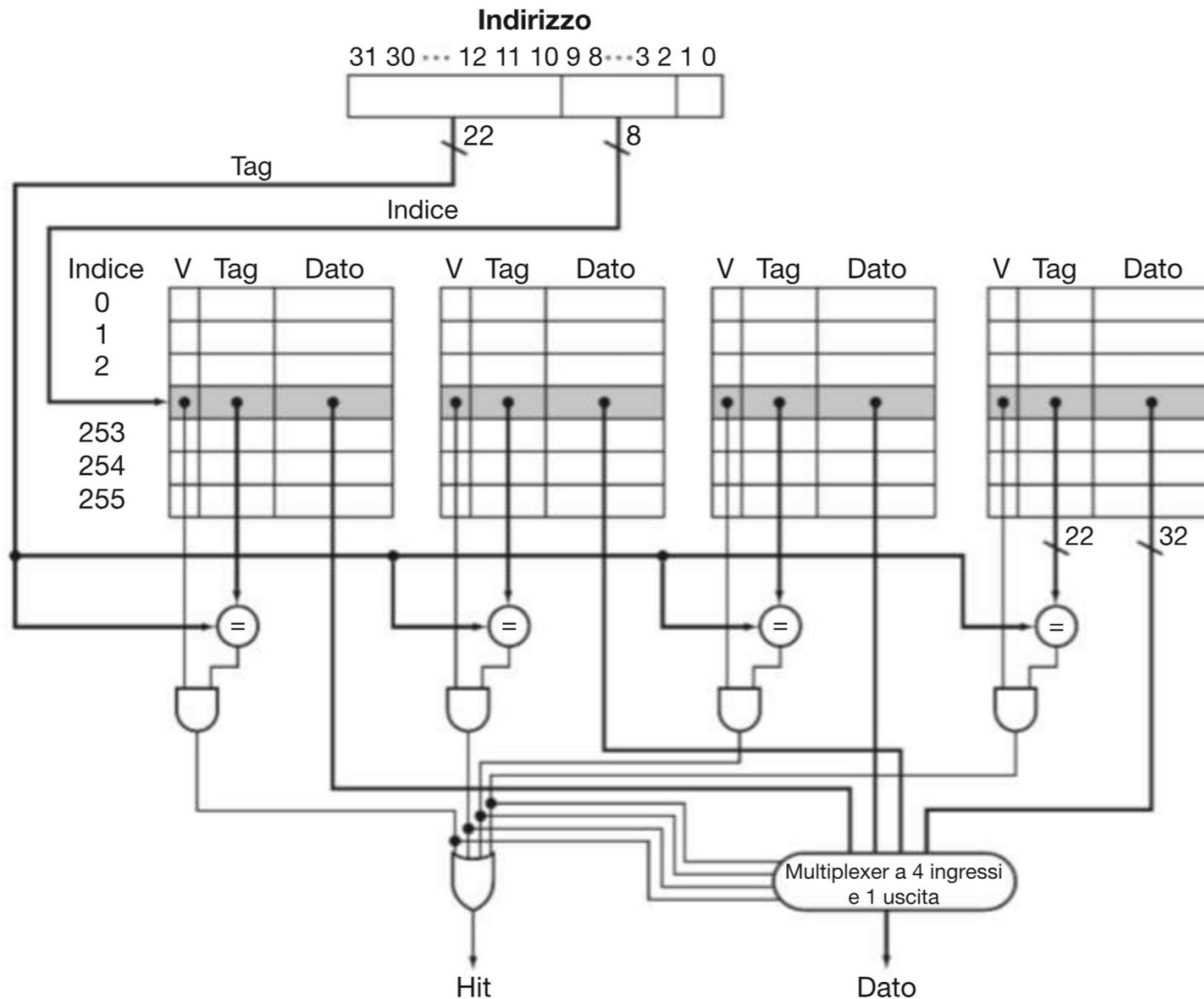
Esempio di prestazione:

Associatività	Frequenza di miss
1	10,3%
2	8,6%
4	8,3%
8	8,1%

- Possiede un numero prestabilito di locazioni alternative (almeno due) nelle quali può essere scritto o letto ogni blocco della memoria principale

(Numero del blocco) modulo (Numero delle **linee** della cache)

Cache set-associativa



Cache set-associativa a quattro vie:

- quattro comparatori
- 1 multiplexer con 4 ingressi e 1 uscita.

I comparatori individuano il blocco di quale insieme, se esiste, corrisponde al tag dell'indirizzo.

L'uscita dei comparatori viene utilizzata per selezionare il dato da uno dei quattro blocchi della linea selezionata, attraverso un multiplexer pilotato dai segnali di hit dei diversi blocchi.

Gestione delle miss

- La gestione di miss e hit necessita di una modifica all'unità di controllo del processore *
- Miss della cache (istruzioni)
 1. inviare il valore originale del program counter alla memoria;
 2. comandare alla memoria principale di eseguire un'operazione di lettura e attendere che la memoria completi la lettura;
 3. scrivere la parola che proviene dalla memoria nella posizione opportuna del blocco della cache, aggiornare il campo tag corrispondente scrivendovi i bit più significativi dell'indirizzo presi direttamente dalla ALU e impostare a 1 il bit di validità;
 4. far ripartire l'esecuzione dell'istruzione dall'inizio, ripetendo la fase di fetch, che questa volta troverà l'istruzione all'interno della cache.
- Miss della cache (dati): sostanzialmente identico

Gestione della scrittura

Write-through

- Ad ogni scrittura in cache si modifica il valore anche in memoria
- Se il dato da scrivere non è presente in cache, viene prima caricato, poi modificato in cache e in memoria
- Prestazioni non buone (uso di un buffer di scrittura)

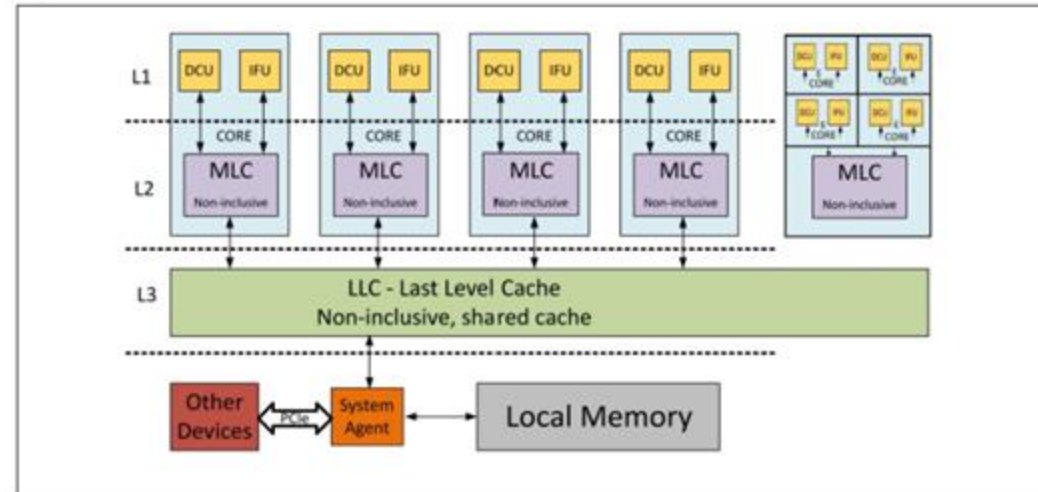
Write-back

- Il dato viene scritto solo in cache
- La scrittura al livello inferiore avviene solo quando il blocco nella cache deve essere rimpiazzato
- Più complesso da implementare

Memoria cache

12th Generation Intel® Core™ Processors (2022)

Hybrid Cache



NOTES

1. L1 Data cache (DCU) - 48KB (P-core) - 32KB (E-Core)
2. L1 Instruction cache (IFU) - 32KB (P-Core) - 64KB (E-Core)
3. MLC - Mid Level Cache - 1.25MB (P-Core) - 2MB (shared by 4 E-Cores)

P Cores 1st level cache is divided into a data cache (DFU) and an instruction cache (IFU). The processor 1st level cache size is 48KB for data and 32KB for instructions. The 1st level cache is an 12-way associative cache.

E Cores 1st level cache is divided into a data cache (DFU) and an instruction cache (IFU). The processor 1st level cache size is 64KB for data and 32KB for instructions. The 1st level cache is an 8-way associative cache.

The 2nd level cache holds both data and instructions. It is also referred to as mid-level cache or MLC. The P processor 2nd level cache size is 1.25MB and is a 10-way non-inclusive associative cache., 4 E Cores processors share 2MB 2nd level cache and is a 16-way non-inclusive. associative cache.

SiFive Performance P550

Memory System And Caches

The P550 memory system has been tuned for high performance workloads. The instruction memory system consists of a dedicated 32KB 4-way set-associative L1 instruction cache with a line size of 64 bytes. The data memory subsystem has a 4-way set-associative 32KB write-back L1 data cache that supports 64-byte cache lines.

The private L2 cache is a 256KB 8-way set-associative cache with a line size of 64 bytes.

A flexible Level 3 cache can be configured to be either 1MB, 2MB or 4MB, with a multi-cluster option of 8MB.